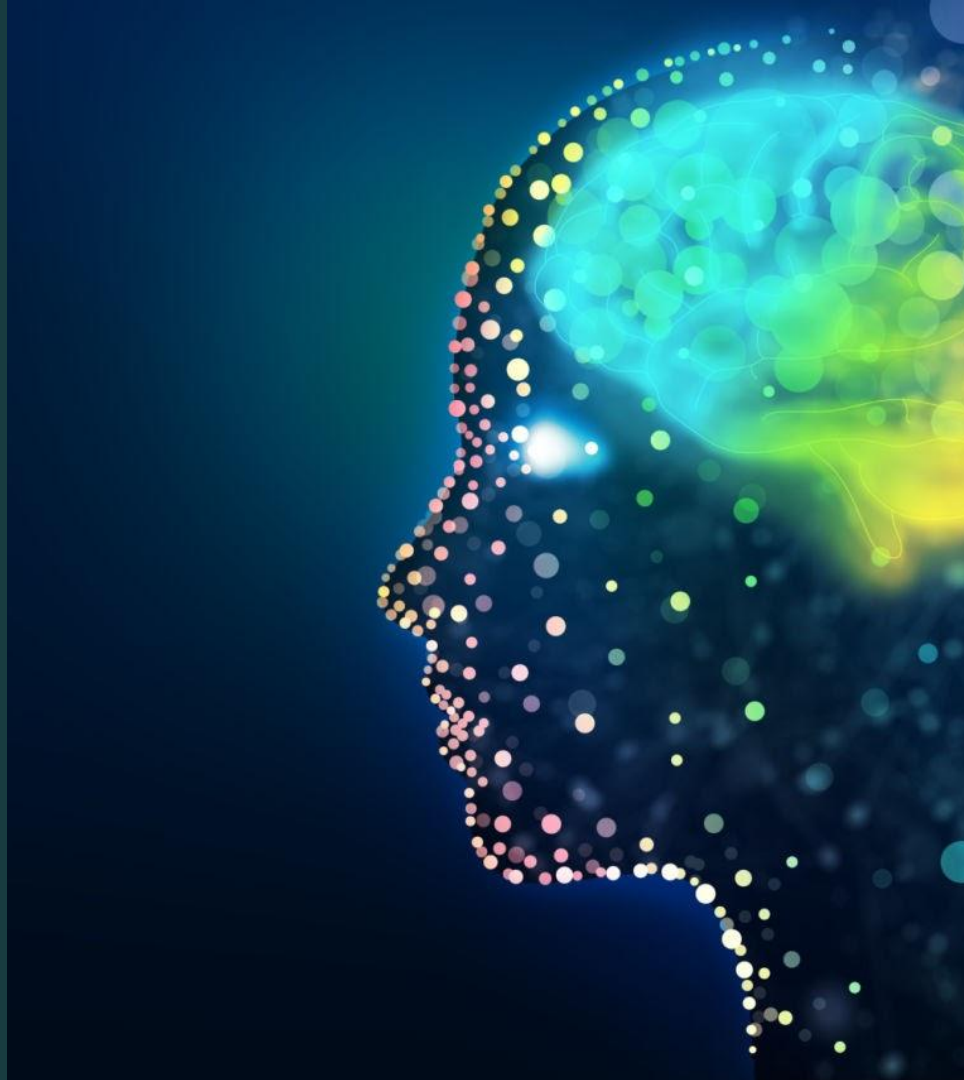
Jacqueline Tang

Facial Recognition Project

How machine learning can bridge emotional gaps



What is this trying to accomplish?

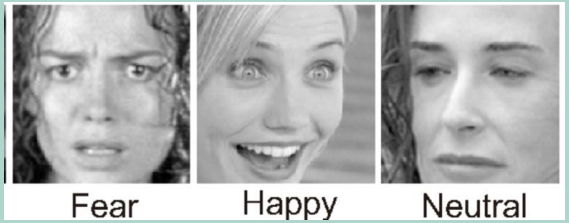
- Help out those with alexithymia (emotional blindness), especially neurodivergent people (non-medical term encompassing a wide variety of mental disorders), to express and communicate emotions
- Can deep-learning be used as a solution? Will neurodivergent people have a better chance at expressing their emotions? Can those emotions be handled better?
- Can caretakers use this technology to help these people?



How can we use a model trained from scratch to help neurodivergent people express their emotions using facial recognition technology?

Solution/Approach

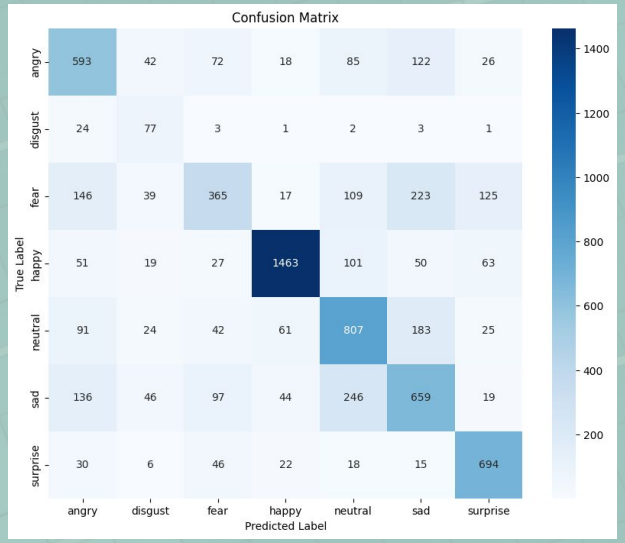
- Test various pretrained models that have been fine-tuned along with a model trained from scratch towards facial emotion recognition by training on labeled dataset.
- Goal: Classify facial images with 7 classes based on expressed emotion.
- Kaggle Dataset used: FER2013 (Predetermined training/testing sets)
- 80/20 Training/Validation split. Testing dataset provided separately.

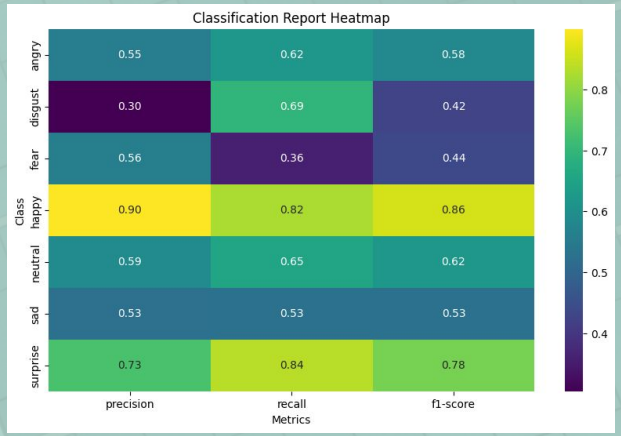


Results

Highest Test Accuracy: 69.713%

- ‘Happy’ in FER2013 had 1774 images but ‘Disgust’ had 111, result of class imbalance.
- ‘Fear’ with high Precision but low Recall → Model misses many true “Fear” labels.
- ‘Disgust’ with high Recall but low Precision → Model tends to incorrectly label other labels as ‘disgust’.
- Best scoring class → ‘Happy’





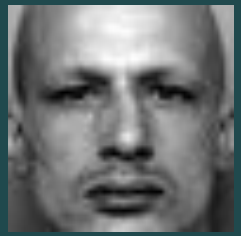
Problems and Limitations.

Class Labels:

- Labels don't necessarily pass the eye test. Overlap between different labels could significantly hamper training.
- Model will struggle with emotions that have less distinguishing features.



True Label: "Angry"



True Label: "Disgust"

Can you tell the difference?

Class Imbalance:

- Training dataset **heavily** imbalanced. 436 "Disgust" samples vs. 4097 "Fear" samples.
- Result → Weak discriminating ability between "Disgust" and other labels.
- Addressed issue by weighting samples based on labels.

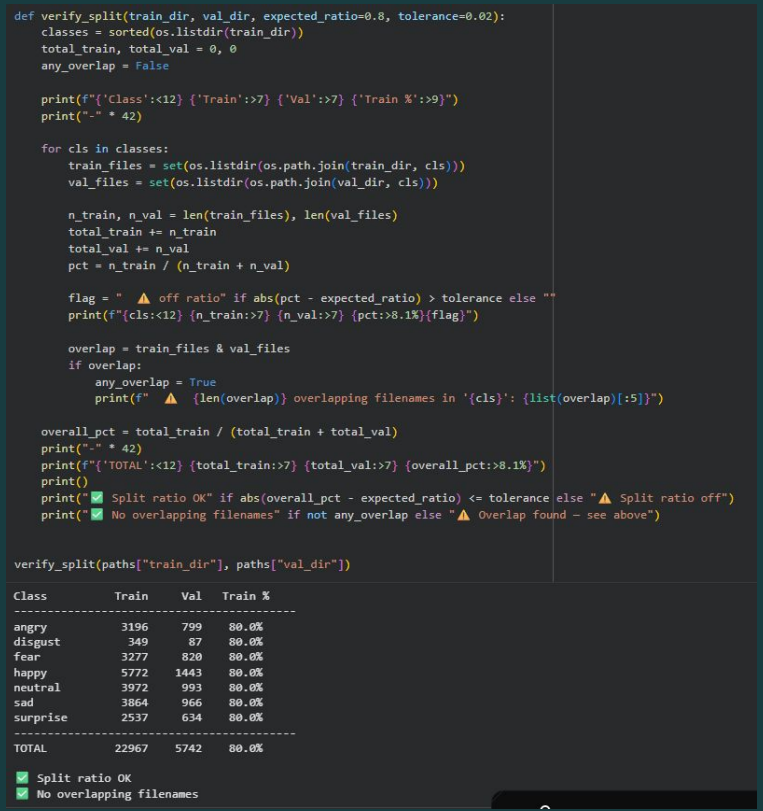
```
angry weight:    1.0266
disgust weight: 9.4011
fear weight:     1.0012
happy weight:    0.5684
neutral weight: 0.8260
sad weight:      0.8491
surprise weight:         1.2933
```

Each class's weight

This could be a bottleneck in our model's performances!

Workflow

- **Data Preprocessing (`01_prepare_data.ipynb`)**:
  - **Training Data**: We perform a 80-20 Stratified Split across all classes for a *training* and *validation* dataset. Test data is provided by the FER2013 Dataset.
  - Saves a **single, unified** set of split data in our drive that is used for all model training.
- **Model Training (`02_train_model.ipynb`)**:
  - Trains one pre-trained model and saves the model weights + train history with early stopping. (Ran multiple times across all models)
  - **Hyperparameters:**
    - Batch Size: 64, Epochs: 30, Learning Rate: 1e-4, Optimizer: Adam, Loss Function: Cross Entropy, Early Stopping Threshold: 7 epochs.
- **Model Evaluation (`03_evaluate.ipynb`)**:
  - Tests all saved model weights against the test dataset provided by FER2013



```
def verify_split(train_dir, val_dir, expected_ratio=0.8, tolerance=0.02):
    classes = sorted(os.listdir(train_dir))
    total_train, total_val = 0, 0
    any_overlap = False

    print(f"{'Class':<12} {'Train':>7} {'Val':>7} {'Train %':>9}")
    print("-" * 42)

    for cls in classes:
        train_files = set(os.listdir(os.path.join(train_dir, cls)))
        val_files = set(os.listdir(os.path.join(val_dir, cls)))

        n_train, n_val = len(train_files), len(val_files)
        total_train += n_train
        total_val += n_val
        pct = n_train / (n_train + n_val)

        flag = "  ⚠️ off ratio" if abs(pct - expected_ratio) > tolerance else ""
        print(f"{cls:<12} {n_train:>7} {n_val:>7} {pct:>8.1%}{flag}")

        overlap = train_files & val_files
        if overlap:
            any_overlap = True
            print(f"  ⚠️  {len(overlap)} overlapping filenames in '{cls}': {list(overlap)[:5]}")

    overall_pct = total_train / (total_train + total_val)
    print("-" * 42)
    print(f"{'TOTAL':<12} {total_train:>7} {total_val:>7} {overall_pct:>8.1%}")
    print()
    print("✅ Split ratio OK" if abs(overall_pct - expected_ratio) <= tolerance else "⚠️ Split ratio off")
    print("✅ No overlapping filenames" if not any_overlap else "⚠️ Overlap found — see above")


verify_split(paths["train_dir"], paths["val_dir"])
```

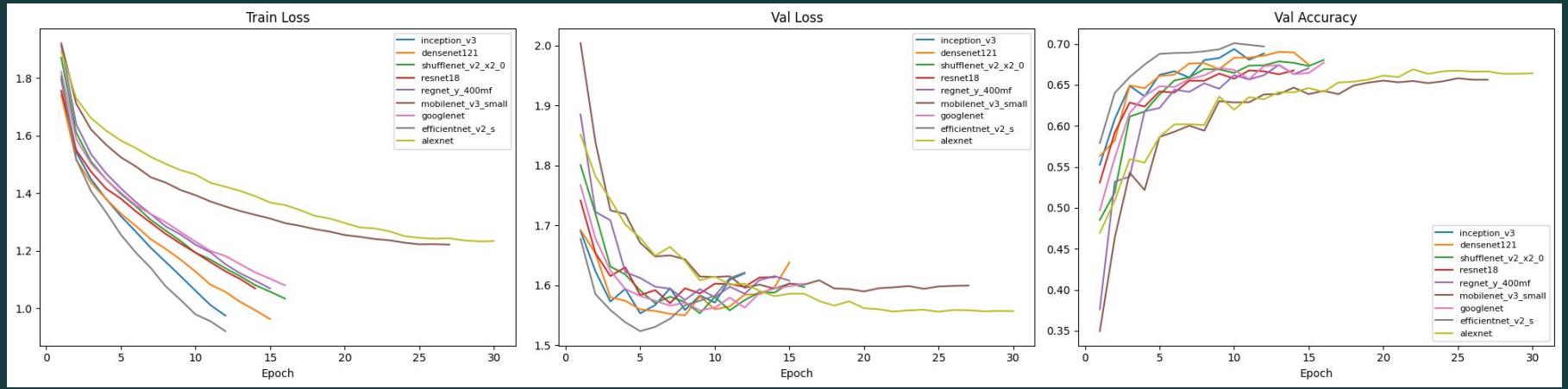
```
Class          Train     Val   Train %
------------------------------------------
angry           3196     799    80.0%
disgust          349      87    80.0%
fear            3277     820    80.0%
happy           5772    1443    80.0%
neutral         3972     993    80.0%
sad             3864     966    80.0%
surprise        2537     634    80.0%
------------------------------------------
TOTAL          22967    5742    80.0%

✅ Split ratio OK
✅ No overlapping filenames
```

Results Analysis Pt. 1

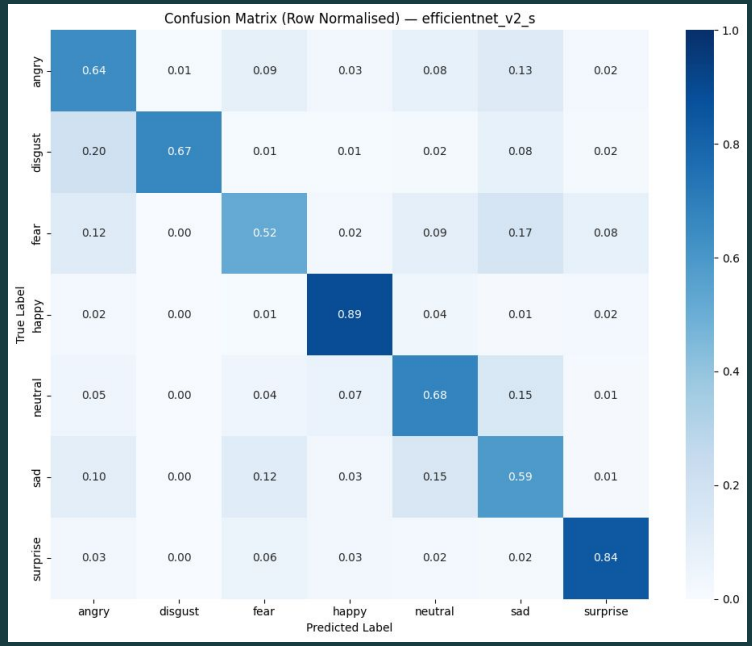
- **Most models converge before 30 epochs:** Most Val Loss/Accuracy plateaus at around 15 epochs, as showing by the early stopping in the training history visualizations.
- **Best Performing Model:** EfficientNet V2 across all metrics: (accuracy, f1, precision, recall)
    - Across all models, no significant difference between accuracy, f1, precision, recall.
- **Worst Performing Model**: Mobilenet_v3_small performed the worst, having an accuracy of 0.658, still indicating strong performance.

	name	accuracy	f1	precision	recall
7	efficientnet_v2_s	0.697130	0.694243	0.693688	0.697130
0	inception_v3	0.693647	0.691294	0.698578	0.693647
6	googlenet	0.676372	0.678268	0.686929	0.676372
4	regnet_y_400mf	0.676094	0.673337	0.679240	0.676094
2	shufflenet_v2_x2_0	0.675536	0.677169	0.682132	0.675536
1	densenet121	0.672053	0.673607	0.679184	0.672053
8	alexnet	0.670242	0.669765	0.672343	0.670242
3	resnet18	0.669546	0.667797	0.674694	0.669546
5	mobilenet_v3_small	0.657565	0.658459	0.665513	0.657565



Results Analysis Pt. 2

- **Generalized Class Performance:**
  - **Happy** and **Surprise** were most correctly labeled by all models: 0.90, 0.83 by efficient net respectively.
  - Most models failed to correctly label **Fear**, (0.51 by efficient net)
- **While EfficientNet V2 had the best test performance overall, it did not score the highest amongst every class/emotion.**
  - Worse-performing models such as MobileNet V3 (.658 accuracy) and ShuffleNet V2 (.676 accuracy) outscored EfficientNet on **Disgust** (0.77, 0.71 vs. 0.62) and others.
  - **→ Cannot concretely say EfficientNet is the best model to use! Dependent on both test dataset (skewed towards happy and fear) as well as use case (which emotions should we prioritize, if not all of them equally?**



Takeaways

- Model accuracies are all in the 65-70% accuracy range.
- A reason for this performance may be due to bottlenecks inherent to the dataset we used:
- May need a dataset with a high amount of images to train on for certain emotions (especially ‘Disgust,’ ‘Surprise,’ ‘Angry’)
- Dangers of privacy concerns since the model stores biological information (e.g. selling that information) due to real faces being fed into the algorithm, certain groups such as corporations and governments should not get a hold

Any questions?

GitHub repository of the code linked here: https://github.com/jt187276/FER2013-Facial-Recognition